

The Spec Writer's Questions

Seventy-eight questions reverse-engineered from eight research documents: universal questions for every spec, riders per part, composition questions per assembly, and the detectors that flag what is not a part — each traceable to the finding it prevents and the gate rule it feeds.

1. Universal questions — asked for every spec

Dimension 1 — Who

U-01. Who uses this, and what are they trying to get done in one sentence each?

Why: [brief] Unstated requirements are followed ~41% of the time; the user's job is the requirement most often left unstated. · *Fills:* `intent.user` · *Gate:* R1

U-02. Is there more than one kind of user (admin vs member vs anonymous)? Name each.

Why: [enf] States a test account cannot reach are unverifiable; every role must be enumerated to be checked. · *Fills:* `substance.permissions` · *Gate:* R1

Dimension 2 — Entry

U-03. What is the exact URL or entry point, and what does the very first screen show before any data exists?

Why: [gate] Empty states are one of the five dimensions speakers always omit as obvious. · *Fills:* `substance-surfaces[0]` · *Gate:* R1

U-04. How does a brand-new user get from arrival to their first successful action — every screen in between?

Why: [parts] Every starter kit ships onboarding because every app needs it and every speaker skips it. · *Fills:* `substance-surfaces` · *Gate:* R1

Dimension 3 — Surfaces

U-05. List every screen. For each: its route, its purpose in one line.

Why: [gate] Surfaces are the largest hole class; the dead-buttons failure began as an unlisted control. · *Fills:* `substance-surfaces` · *Gate:* R1

U-06. For every control on every screen: what endpoint does it call — or is it deliberately display-only?

Why: [gate] R5: a control with neither is a rejection, not a warning. There is no third option. · *Fills:* `substance-surfaces[]-controls` · *Gate:* R5

U-07. Will two copies of the same component ever appear on one page? If yes, what distinguishes their names?

Why: [parts] Flat naming breaks composition (Salesforce namespaces exist for this; web components' global registry is the counter-example). · *Fills:* `substance-surfaces[]-controls[]-label` · *Gate:* R5

Dimension 4 — Actions

U-08. For each control: what does the user see on success, on failure, while loading, and when there is nothing to show?

Why: [gate] The four states speakers never mention; [parts] Playwright/Maestro auto-wait exists because loading is universal. · *Fills:* `substance-surfaces[]-controls` · *Gate:* R1

U-09. Which actions change something outside this app (send, charge, delete, post)? For each: is it reversible, and how?

Why: [gate] R11; [nova] retries re-execute — an irreversible action without rollback doubles on retry. · *Fills:* `substance-actions` · *Gate:* R11

U-10. Which actions must a human approve before they run?

Why: [nova] Park-by-default for side effects; approval is a field, not a vibe. · *Fills:* `substance.actions[].needs_approval` · *Gate:* R11

Dimension 5 — Data

U-11. What is stored? For each thing: its fields, where it lives (exact store), and who can read and write it.

Why: [gate] R12; [enf] a criterion over a row needs the row's address to go and look. · *Fills:* `substance.data` · *Gate:* R12

U-12. What happens to the data when a record's parent is deleted — cascade, orphan, or forbid?

Why: [parts] Lifecycle is in every CRUD platform's data model and no speaker's description. · *Fills:* `substance.data[].lifecycle` · *Gate:* R1

Dimension 6 — External

U-13. List every outside service. For each: what it is for, what credential it needs, WHO supplies that credential, and WHERE they paste it.

Why: [gate] R6 — credential custody is the classic silently-assumed answer. · *Fills:* `substance.externals` · *Gate:* R6

U-14. For each external: what should the app do when that service is down — block, degrade, or queue?

Why: [enf] 'With the backend stopped...' criteria exist only if the degraded behaviour was ever decided. · *Fills:* `substance.externals[].on_outage` · *Gate:* R1

Dimension 7 — Identity

U-15. How does the system know who the user is — and how long does that hold (session length, remember-me, logout)?

Why: [parts] Auth is the most universal part (11/14 sources) and session policy is its most-skipped config. · *Fills:* `parts.auth.config` · *Gate:* R1

Dimension 8 — Permissions

U-16. For each role: name one thing it **MUST** be able to do and one thing it **MUST NOT** — per screen it touches.

Why: [enf] Authorisation matrices are machine-checkable only for enumerated role-endpoint pairs; unenumerated = unverifiable. · *Fills:* `substance.permissions` · *Gate:* R1

Dimension 9 — Errors

U-17. For the three most likely failures (server down, bad input, no permission): what exact words does the user see?

Why: [gate] Errors are a top-five hole; exact strings make the criterion mechanical ('shows "Cannot reach server" within 5s'). · *Fills:* `substance.surfaces[ ].states.error` · *Gate:* R3

Dimension 10 — Empty states

U-18. Before any data exists, what does each list, table and dashboard show, and what does it tell the user to do?

Why: [gate] Empty is a state, not an accident; unspecified empty states become blank panels. · *Fills:* `substance.surfaces[ ].states.empty` · *Gate:* R1

Dimension 11 — Money and limits

U-19. What are the ceilings — spend, records, file size, requests per minute, retry attempts, timeouts?

Why: [parts] Bubble's documented envelope limits (300s workflows, 50k sorted records) are where composed blocks silently break; declare yours. · *Fills:* `bounds` · *Gate:* R1

U-20. What is the build budget (money, time or tokens), and what should happen when it is hit?

Why: [gate] Budget is a required field; [nova] hard stops with partial results beat silent overruns. · *Fills:* `bounds.budget` · *Gate:* R1

Dimension 12 — Environment

U-21. Where does this run — host, process, port — and how is it started and restarted?

Why: [gate] Environment is required; [enf] a verifier needs a reachable URL, not a description. · *Fills:* `bounds.environment` · *Gate:* R1

U-22. Is there a test tenant or test account the verifier may use on the live system, and what marks its traffic as synthetic?

Why: [enf] Destructive controls cannot be click-proven on production without a test tenant + cleanup contract; the synthetic-traffic marker is the documented pattern. · *Fills:* `bounds.environment.test_tenancy` · *Gate:* R10

Dimension 13 — Rollback

U-23. If the deployment breaks, what is the named way back — and who may pull that lever?

Why: [gate] R11 at spec level; 'git revert' is an answer, 'we'll figure it out' is not. · *Fills:* `bounds.rollback` · *Gate:* R11

Dimension 14 — Acceptance

U-24. For each thing you just said, what observation proves it — a command and its output, a response body, a row, a message in a mailbox?

Why: [enf] The six-field tuple: a criterion needs precondition, trigger, channel+address, extraction, decision rule, and time budget. · *Fills:* `gate.acceptance` · *Gate:* R3

U-25. For every outcome that is NOT on screen (email, row, webhook, queued job): where exactly does a script go to look, and how long may it wait?

Why: [parts] The invisible-outcome gap is the weakest area of all existing formats; the channel must be declared or the criterion is undecidable. · *Fills:* `gate.acceptance[].observe` · *Gate:* R3

U-26. Which criteria genuinely need a human eye? (Fewer is better; each must name the exact evidence the human looks at.)

Why: [gate] human:true criteria are counted separately; [enf] the human judges diffs and named evidence packs, never whole pages. · *Fills:* `gate.acceptance[ ].human` · *Gate:* R3

Dimension 15 — Out of scope

U-27. Name at least three things a reasonable person might assume is included that you are NOT building.

Why: [gate] R9 — everything not listed is assumed in scope; an empty boundary is never true. · *Fills:* `intent.out_of_scope` · *Gate:* R9

U-28. Of the things now in scope: which are load-bearing (failure blocks release) and which are nice-to-have?

Why: [nova] Load-bearing gates escalation and retry policy; unlabelled criteria all gate equally, which means none really do. · *Fills:* `gate.acceptance[ ].load_bearing` · *Gate:* R3

2. Per-part riders — asked when the part is selected

Questions marked [**channel**] establish an observation channel the assembled app must provide so the criterion is verifiable — the invisible-outcome fix.

Auth

A-01. How do users sign in — email+password, magic link, Google/Microsoft SSO, or several? List all.

Why: [parts] Every starter ships the same quartet; the chosen subset changes every criterion. · *Gate:* R1

A-02. Is self-registration open, invite-only, or admin-created?

Why: [parts] Changes the entry flow and the seeded-test-user precondition. · *Gate:* R1

A-03. Is password reset on? If yes: what address sends the email, and what does its subject line say?

Why: [parts] active_when in action: the reset-email criterion exists only if this is true; the subject string makes it mechanical. · *Gate:* R3

A-04. How long does a session last, and what does the user see the moment it expires mid-task?

Why: [parts] Session policy is auth's most-skipped config; expiry mid-form is the classic unhandled state. · *Gate:* R1

A-05. Is 2FA required, optional, or off?

Why: [parts] Jetstream ships it as a toggle; a toggle unanswered is a criterion undefined. · *Gate:* R1

A-06. After login, where does each role land?

Why: [gate] The success observation of the login criterion is this exact redirect. · *Gate:* R3

A-07. [channel] For verification: what seeded test user exists per role, and who maintains those credentials?

Why: [enf] Every login criterion's precondition; states a test account can't reach are unverifiable. · *Gate:* R10

Records

R-01. Name each record type and its fields — every field, its type, and which are required.

Why: [parts] The domain data model is always business-specific; it cannot be defaulted, only asked. · *Gate:* R12

R-02. Who can create, see, edit and delete each record type? (Owner-only? Team? Everyone?)

Why: [enf] The enumerated authz matrix is the only checkable form of permissions. · *Gate:* R1

R-03. Is delete soft (recoverable) or hard? If hard, say so out loud.

Why: [gate] R11 — hard delete is irreversible and needs its rollback named or accepted. ·
Gate: R11

R-04. How many records before the list needs search, filters or pagination — and which of those are you asking for?

Why: [parts] Envelope limits: composed blocks break at scale thresholds nobody declared (Bubble's 50k sort cap). · *Gate:* R1

R-05. Can records be exported? In what format, and does the export include everything or a subset?

Why: [parts] Export is in every CRUD platform and almost no spoken description. ·
Gate: R1

Forms

F-01. For each form field: what makes an input invalid, and what exact message does the user see?

Why: [gate] Validation strings make error criteria mechanical; 'proper validation' is a banned-word answer. · *Gate:* R4

F-02. What happens on submit — where does the data go, and what does the user see next?

Why: [gate] R5/R12 — the endpoint and the storage location are the two mandatory addresses. · *Gate:* R5

F-03. Can a half-finished form be saved or is it lost on navigation? Say which.

Why: [parts] Draft state is a universal silent assumption in both directions. · *Gate:* R1

F-04. Can the same form be submitted twice by double-click or refresh? What prevents a duplicate?

Why: [nova] Idempotency keys exist because retries double side effects; forms are where users retry. · *Gate:* R11

Permissions

P-01. List the roles. For each pair (role, screen): allowed or not? For each pair (role, action): allowed or not?

Why: [enf] Only enumerated pairs are checkable; the matrix IS the spec. · *Gate:* R1

P-02. Who assigns roles, and can a user ever change their own?

Why: [parts] Privilege escalation by self-service is the classic silent default. · *Gate:* R1

P-03. What does a user see when they hit something they may not do — invisible, disabled, or an error page?

Why: [gate] The deny-path is a surface state; three different builds satisfy 'has permissions' unless this is pinned. · *Gate:* R3

Notify

N-01. Which channels — email, in-app, SMS, push — and for which events exactly?

Why: [parts] Novu ships all four; each event×channel pair is a separate criterion. · *Gate:* R1

N-02. What address or number do messages come FROM, and who owns that domain's sending reputation (DKIM/SPF setup)?

Why: [parts] WP Mail SMTP's 4M installs exist because unsent email is the default; custody question per R6. · *Gate:* R6

N-03. Can users turn notifications off, per channel or per event?

Why: [parts] Preference centres are in every notification service; unasked means unbuilt or uncriterioned. · *Gate:* R1

N-04. [channel] For verification: where may the checker read sent messages (mail-capture endpoint in test tenancy), and how long may delivery take?

Why: [parts] The invisible-outcome gap; the mailbox address and the time budget make 'the email arrives' decidable. · *Gate:* R3

Files

FI-01. What file types and maximum size are accepted, and what does the user see when they exceed them?

Why: [parts] Every upload service leads with these two limits; the rejection message is the criterion string. · *Gate:* R1

FI-02. Where do files physically live (bucket/store, region), and who can fetch a file's URL — is it public, signed, or authenticated?

Why: [gate] R12 plus the most common real-world leak: public buckets nobody decided on. · *Gate:* R12

FI-03. When a record is deleted, do its files go too?

Why: [parts] Lifecycle again; orphaned files are the default outcome of silence. · *Gate:* R1

Reports

RP-01. For each chart or number on the dashboard: what is the exact query behind it, in words?

Why: [enf] A handler that runs and computes the wrong answer passes unless the criterion pins the answer; pin it at spec time. · *Gate:* R3

RP-02. How fresh must the numbers be — live, minutes, or daily?

Why: [enf] The time budget field; 'live' and 'daily batch' are different architectures and different criteria. · *Gate:* R3

RP-03. What does the dashboard show when there is no data yet — and is a zero a zero or a blank?

Why: [gate] Empty states, dashboard edition; blank-vs-zero is a real decision. · *Gate:* R1

Flow

W-01. For each approval or background job: what starts it, who or what approves it, and what happens on approve and on reject?

Why: [parts] 22 of Budibase's 23 templates are this shape; org approval semantics are always business-specific and must be asked, never defaulted. · *Gate:* R1

W-02. How long may a job run or wait before it is considered stuck, and what happens then — retry, alert, or park?

Why: [nova] Every loop needs a Rail-side bound; [parts] Bubble's 300s timeout is the envelope example. · *Gate:* R1

W-03. If the same job fires twice, is that harmless? If not, what makes it fire exactly once?

Why: [nova] Idempotency; queue redelivery is normal, not exceptional. · *Gate:* R11

W-04. [channel] For verification: where can a script see a job's state (queued/running/done) — an admin endpoint, a table, or a span?

Why: [parts] Invisible-outcome channel for the queue; Tracetest-style spans or a status endpoint, but it must be named. · *Gate:* R3

Billing

B-01. Which provider — Stripe, Lemon Squeezy, other — and whose account? Who holds the API keys and where are they pasted?

Why: [gate] R6 custody; [parts] provider choice changes every webhook and criterion. · *Gate:* R6

B-02. One-off payments, subscriptions, or both? If subscriptions: which plans, prices and intervals, exactly?

Why: [parts] The starters split precisely here; a criterion cannot assert an unnamed price. · *Gate:* R1

B-03. What happens in the app the moment payment succeeds — and what happens if the webhook confirming it never arrives?

Why: [parts] Stripe documents that event order is not guaranteed; assert eventual states, never sequences. · *Gate:* R3

B-04. Refunds: who can issue one, from where, and what does it change in the app?

Why: [gate] R11 — a charge is a side effect; the refund is its rollback and must be named.
· *Gate:* R11

B-05. Failed payments and dunning: retry how many times, then what — downgrade, lock, or email?

Why: [parts] Every billing service has a dunning setting; unset means undefined app behaviour on the most common billing event. · *Gate:* R1

B-06. [channel] For verification: is provider test-mode available, and what is the webhook endpoint the checker may hit with test events?

Why: [enf] `stripe listen` /test-mode is the interposed channel; without it billing criteria cannot run against the live system safely. · *Gate:* R10

B-07. Tax and receipts: does the provider handle tax (merchant-of-record) or do you, and must a receipt email go out?

Why: [parts] The Stripe/Lemon Squeezy consolidation is exactly this split; a receipt email is an invisible outcome needing a channel. · *Gate:* R1

3. Composition questions — asked once per assembly

C-01. Which parts are in this assembly, and does any appear twice? Give each instance a short name.

Why: [parts] Instance namespacing is structural, not conventional; two Forms on one page must be distinct by construction. · *Gate:* R1

C-02. What does each part need from another (login before billing? records before reports?)? State every such dependency.

Why: [parts] Provides/requires is the composition rule; undeclared dependencies become ordering bugs. · *Gate:* R1

C-03. Is any part's configuration in conflict with another's (auth says invite-only, billing says self-serve signup)?

Why: [brief] Constraint conflicts collapse compliance; catch the impossible pair at spec time. · *Gate:* R1

C-04. Do two parts write the same data? Who wins?

Why: [parts] Shared-substrate mutation is a documented seam failure (Shopify's sandboxing exists because of it). · *Gate:* R12

C-05. Summed envelopes: do the parts' combined limits fit the declared ceilings (jobs × runtime, records × queries, files × storage)?

Why: [parts] Two individually working blocks fail when composed at scale; sum the envelopes before building. · *Gate:* R1

C-06. One user journey that crosses at least three parts, end to end: walk it. Where does state hand over?

Why: [parts] The seams are where composition breaks; a walked journey surfaces the handovers no per-part question can. · *Gate:* R3

4. Custom-build detectors — the answers that mean 'this is not a part'

X-01. Is there a calculation, matching rule, score or price that is YOUR way of doing it — the thing the business actually is?

Why: [parts] OutSystems' own stepdown rule: 'your own algorithms' means leave the blocks. Vendors agree the algorithm is never in the box.

If yes: Mark as custom scope with its own acceptance criteria; do not force into a part.

X-02. Does any approval flow have rules beyond who-approves (thresholds, delegation, escalation chains)?

Why: [parts] Org approval semantics are always business-specific; parts ship thin defaults only.

If yes: Custom flow logic; the Flow part hosts it but its rules are authored, not configured.

X-03. Does any single unit of work exceed a part's envelope (minutes-long jobs, >tens-of-thousands of records in one view)?

Why: [parts] The envelope stopping rule; composition stops where the block's ceiling is.

If yes: Custom build for that unit; keep parts for the rest.

X-04. Does the customer's description keep using a word no part has (their domain noun)? What is its data shape and behaviour?

Why: [parts] The domain model is the never-templated residue; the unmatched noun is its tell.

If yes: It is the Records part's schema at minimum, custom logic at most — ask R-01 about it first.

How it runs

The gap scan (Spec Builder step 3) asks these in order: universal, then the rider for each selected part, then composition, then the detectors. Any question the conversation already answered is marked covered with its transcript line; anything else becomes an **[ASK]** carrying the question text verbatim — silence is never an answer. The bank is data, not prose: adding a part means adding its rider entries, and the gate rule ids keep every question honest about which mechanical check it feeds. The channel questions are the ones that make invisible outcomes decidable: they force the spec to name where a script goes to look and how long it may wait, before anything is built.